Programare avansată pe obiecte -proiect

Fiecare student va lucra la un proiect individual. Proiectul este structurat în mai multe etape. Scopul este să aplicați conceptele de programare orientată pe obiecte, colecții și persistență, într-un sistem coerent (aplicație în consolă).

Condiții minime pentru punctare

- Proiectul nu prezintă erori de compilare și rulează complet (fără crash la scenariile uzuale).
- Sunt implementate cerințele etapei predate (nu doar schelet).
- Codul este organizat: pachete, nume sugestive, clase/servicii separate, fără duplicări evidente.
- Datele de intrare sunt validate minimal (ex: nu acceptați valori invalide fără mesaj).

Termene de predare

Etapa I	Săptămâna (6 - 10 aprilie 2026)
Etapa II	Săptămâna (1 - 5 iunie 2026)
Etapa III (bonus)	Săptămâna (1 - 5 iunie 2026)

Etapa I

1) Definirea sistemului

Alegeți o temă (din lista sugerată sau propuneți una proprie) și definiți:

- O listă cu cel puțin 12 acțiuni/interogări pe care utilizatorul le poate face în sistem (ex: adăugare, căutare, filtrare, raport).
- O listă cu cel puțin 10 tipuri de obiecte/entități (clase) relevante pentru temă.
- Relații între entități (asociere/compunere) și cel puțin o ierarhie de moștenire.

2) Implementare (aplicație Java - consolă)

- Clase simple cu atribute private/protected, constructori și metode de acces (get/set) unde este cazul.
- Cel puțin 3 colecții diferite pentru gestionarea obiectelor (ex: List, Set, Map).
- Cel puțin o colecție sortată (ex: TreeSet/TreeMap sau sortare cu Comparator).

- Utilizare de moștenire pentru clase adiționale și folosirea lor în colecții.
- Cel puțin o interfață (ex: Printable/Calculable) implementată de una sau mai multe clase.
- Cel puțin un enum (ex: status, tip, categorie) folosit în model.
- Gestionarea excepțiilor (try/catch) pentru scenarii uzuale (ex: element inexistent, input invalid).
- O clasă Service (sau mai multe) care expun operațiile sistemului.
- O clasă Main care orchestrează meniul și apelează serviciile.

Etapa II

Extindeți proiectul din prima etapă prin realizarea persistenței utilizând o bază de date relațională și JDBC.

1) Persistență și servicii CRUD

- Modelați schema bazei de date (tabele + chei) pentru entitățile principale.
- Implementați operații de tip Create/Read/Update/Delete pentru cel puțin 5 clase (entități).
- Realizați un strat DAO/Repository (recomandat) separat de stratul Service.
- Implementați servicii singleton (poate fi generic) pentru citire/scriere din baza de date.

2) Serviciu de audit

Realizați un serviciu care scrie într-un fișier CSV de fiecare dată când este executată una dintre acțiunile descrise în etapa I.

Structura fișierului:

nume_actiune, timestamp

Recomandare: folosiți format ISO-8601 pentru timestamp (ex: 2026-04-02T14:23:11).

Etapa III (bonus)

Îmbunătățiri pentru puncte suplimentare (alegeți cel puțin 2):

- Unit testing (JUnit): minim 8 teste relevante (servicii/validări).
- Folosirea Streams API pentru cel puțin 5 acțiuni (filtrare, map, reduce, groupingBy).
- Serializare/Export (CSV/JSON) pentru rapoarte (în afară de audit).
- Logare (java.util.logging / SLF4J) în loc de System.out pentru mesaje tehnice.
- Documentare: README cu pași de rulare + schema BD + exemple de comenzi din meniu.

Teme sugerate (2026)

1) platformă de rezervări (restaurant/sală sport) - clienți, locații, rezervări, plăți
2) sistem de management evenimente - evenimente, bilete, participanți, locații, check-in
3) aplicație de tracking obiceiuri/fitness - utilizatori, obiective, antrenamente, statistici
4) bibliotecă digitală - cărți, autori, împrumuturi, recenzii, categorii
5) catalog academic extins - studenți, cursuri, profesori, note, prezențe, orar
6) gestiune inventar depozit - produse, loturi, furnizori, recepții, livrări
7) platformă de e-learning - cursuri, module, quiz-uri, înscrieri, progres
8) sistem de licitații - licitații, oferte, produse, utilizatori, statusuri
9) food delivery - restaurante, meniuri, comenzi, curieri, rute
10) helpdesk / ticketing - utilizatori, tichete, categorii, priorități, SLA
11) smart home manager - dispozitive, camere, automatizări, alerte, consum
12) aplicație de buget personal - conturi, tranzacții, categorii, rapoarte, bugete)